Ejercicio 04: clasificador visual con CNN y Model Card

INF-8239 Ciencia de Datos II, Unidad 02 (U02.E04)

Estudiante	Roque Alexis Ventura Vallejo
Matrícula	100805873
Repositorio	github.com/RogueVentura/UASD-Maestria-Ciencia-De-Datos, Ciencia de datos II/Unidad II/Ejercicio 04
Base	LAB07, proyecto INF8239_U02_CV_Proyecto_Base (uv, TensorFlow en entorno separado)

1. Dataset y problema

Trabajé con Fashion-MNIST (Xiao, Rasul y Vollgraf, 2017; licencia MIT): 70,000 imágenes de prendas en escala de grises de 28×28 píxeles, divididas por sus autores en 60,000 de entrenamiento y 10,000 de prueba, con 10 clases. TensorFlow lo descarga a su caché con *tf.keras.datasets.fashion_mnist*, así que no se sube al repositorio. La tarea es comparar, con la misma partición, un baseline denso y una CNN, y decidir si la mejora predictiva, si la hay, justifica el costo computacional.

2. Auditoría y partición reproducible

La partición es fija y no depende de una semilla: las primeras 54,000 imágenes del conjunto de entrenamiento se usan para entrenar, las últimas 6,000 para validación (EarlyStopping y checkpoint) y las 10,000 de prueba solo para la evaluación final. La función *audit_splits* calcula un hash MD5 de los píxeles de cada imagen para buscar duplicados exactos dentro de cada partición y entre particiones. El resultado se guarda en *reports/audit.json*:

Partición	Forma	Rango	Imágenes por clase	Duplicados	Compartidas
Entrenamiento	54,000 × 28 × 28 × 1	[0, 1]	5,367 a 5,445	0	con validación: 0
Validación	6,000 × 28 × 28 × 1	[0, 1]	555 a 633	0	con prueba: 0
Prueba	10,000 × 28 × 28 × 1	[0, 1]	1,000 exactas	0	con entrenamiento: 0

No hay fuga por imágenes repetidas entre particiones y la prueba está balanceada, así que el F1-macro y la accuracy casi coinciden. La validación no es estratificada: varía entre 555 (Sneaker) y 633 (Coat) imágenes por clase, una diferencia que no afecta la selección del mejor punto. El preprocesamiento divide los píxeles entre 255 y añade el canal, y *validate_images* rechaza formas, rangos o etiquetas fuera de contrato antes de entrenar. Con semilla 42 y TF_DETERMINISTIC_OPS=1, cuatro corridas en el mismo equipo dieron el mismo F1 hasta el último decimal.

3. Baseline y CNN

Ambos modelos usan Adam, entropía cruzada, lotes de 128, un máximo de 8 épocas y EarlyStopping con paciencia 2 que restaura los mejores pesos según la pérdida de validación.

Modelo	Arquitectura	Parámetros
Baseline denso	Flatten, Dense(64, ReLU), Dense(10, softmax)	50,890
CNN	Conv2D(32, 3×3), MaxPool, Conv2D(64, 3×3), MaxPool, GlobalAveragePooling, Dropout(0.25), Dense(10, softmax)	19,466

4. Métricas, curvas y errores por clase

Modelo (prueba, 10,000 imágenes)	F1-macro	Accuracy	Época con menor pérdida de validación
Baseline denso, 8 épocas	0.864	0.865	8 de 8
CNN, 8 épocas	0.789	0.793	8 de 8
Baseline denso, 30 épocas (sensibilidad)	0.875	0.876	17 (paró en 19)
CNN, 30 épocas (sensibilidad)	0.848	0.851	30 de 30

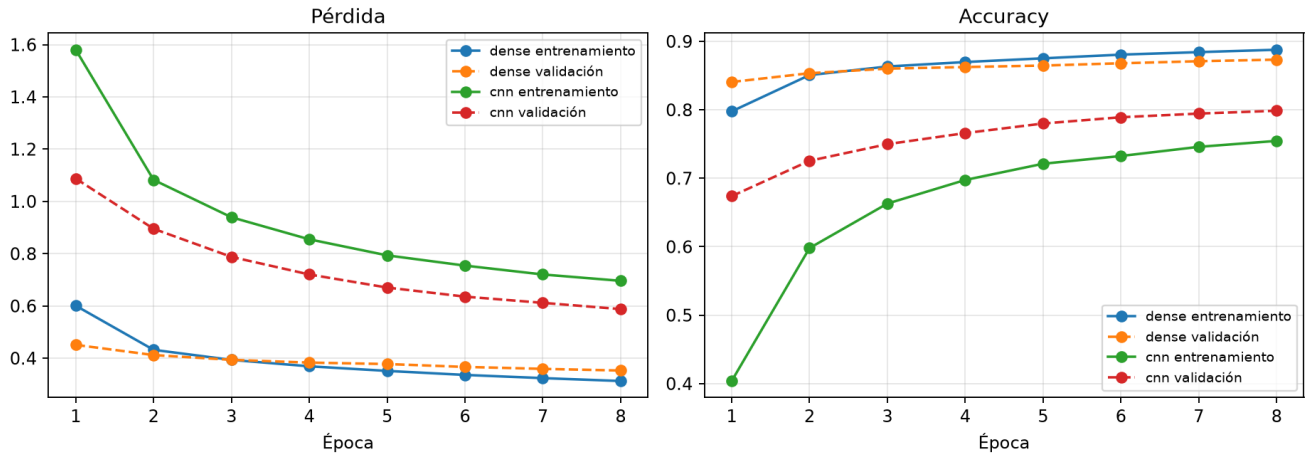
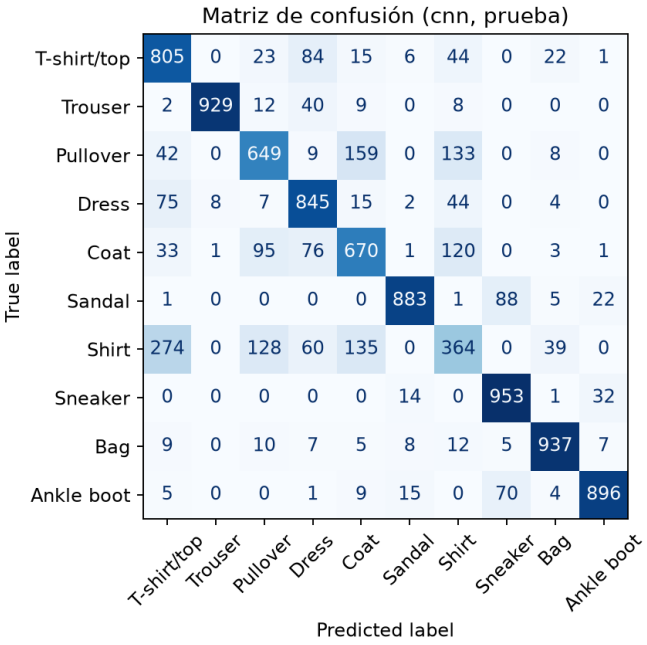


Figura 1. Pérdida y accuracy por época, entrenamiento (línea continua) y validación (discontinua), corrida de 8 épocas.

En el baseline denso las dos pérdidas bajan juntas y la brecha es pequeña. En la CNN ambas pérdidas siguen bajando en la última época y la accuracy de validación (0.799) supera a la de entrenamiento (0.755), porque el Dropout solo actúa al entrenar. Es un caso de subajuste, no de sobreajuste: el mejor punto es siempre la última época. Por eso repetí el entrenamiento con 30 épocas. La CNN subió a 0.848, pero siguió por debajo del denso (0.875).

El promedio oculta una clase débil. La tabla de la derecha da el F1 de cada clase en prueba (8 épocas) junto a la matriz de confusión de la CNN:



Clase	F1 denso	F1 CNN	Recall CNN
T-shirt/top	0.82	0.72	0.81
Trouser	0.97	0.96	0.93
Pullover	0.75	0.67	0.65
Dress	0.86	0.80	0.84
Coat	0.78	0.66	0.67
Sandal	0.95	0.92	0.88
Shirt	0.65	0.42	0.36
Sneaker	0.93	0.90	0.95
Bag	0.96	0.93	0.94
Ankle boot	0.95	0.91	0.90

Figura 2. Matriz de confusión de la CNN en prueba (filas: clase real).

Shirt es la clase más difícil para los dos modelos: la CNN solo recupera el 36 % de las camisas (F1 0.42). Los pares que más se confunden son Shirt con T-shirt/top (274 casos), Pullover con Coat (159), Shirt con Coat (135), Pullover con Shirt (133) y Shirt con Pullover (128). Son prendas de torso con la misma silueta a 28×28 píxeles. En el otro extremo están Trouser, Bag, Sandal, Sneaker y Ankle boot, con F1 de 0.90 o más.

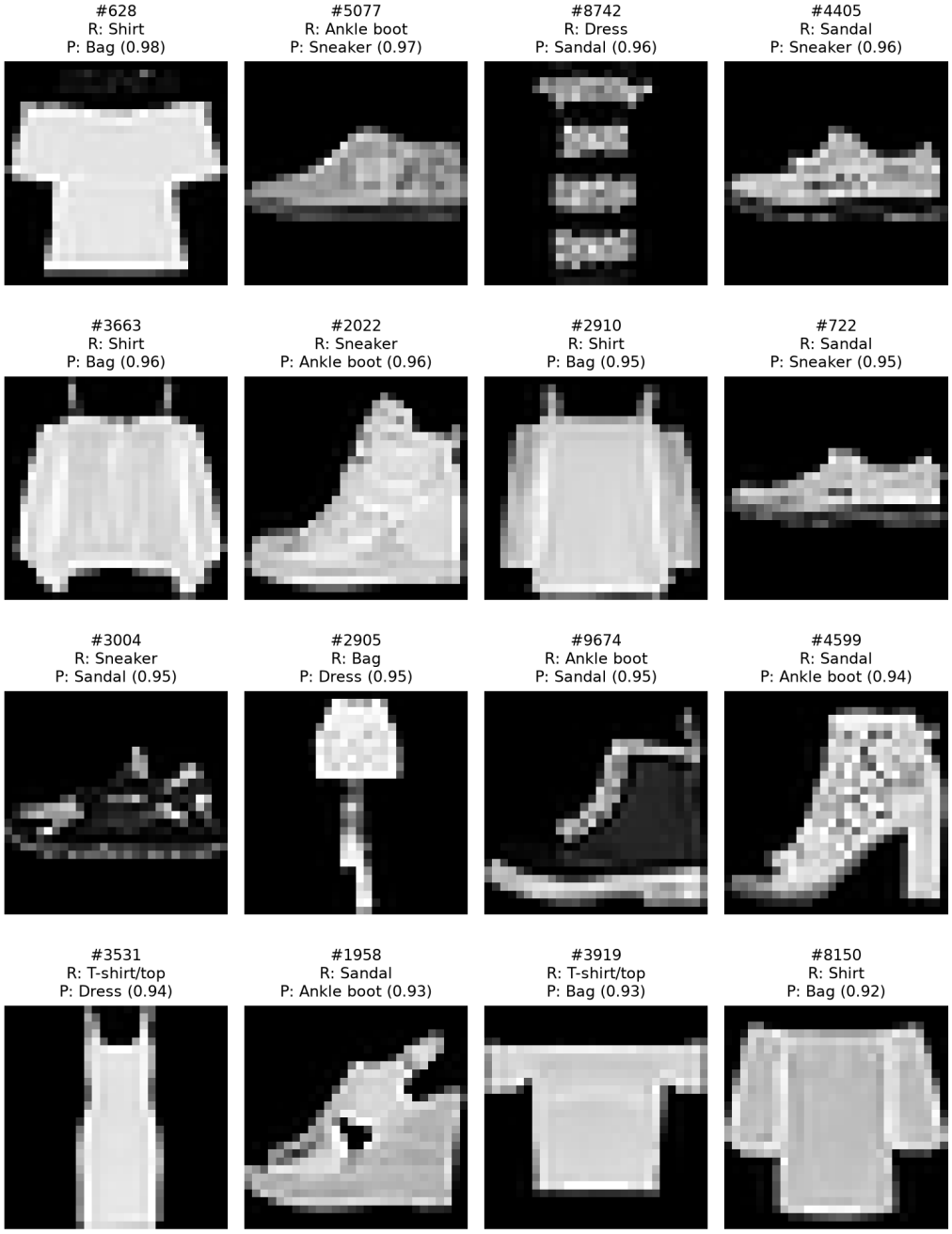


Figura 3. Los 16 errores de la CNN con mayor confianza (#índice de prueba, R: clase real, P: predicha y su probabilidad).

Revisé los 16 errores de mayor confianza (de 0.92 a 0.98). Hay tres grupos: (1) camisas anchas o con tirantes clasificadas como Bag, cuya silueta rectangular se parece a la de un bolso; (2) confusiones entre calzado, como sandalias cerradas predichas como Sneaker y botines como Sandal; y (3) imágenes ambiguas o con etiqueta dudosa, como la #8742 (etiquetada Dress, sin forma de vestido) y la #2905 (etiquetada Bag, con silueta de vestido). La confianza alta en errores muestra que la probabilidad softmax no está calibrada y no debe leerse como certeza.

5. Costo, tamaño y decisión técnica

Medida (8 épocas, CPU)	Baseline denso	CNN	Relación CNN / denso
Parámetros	50,890	19,466	0.38
Archivo .keras	619 KB	262 KB	0.42
Tiempo de entrenamiento	5.8 s	66.9 s	11.5
Inferencia, mediana de 5 repeticiones	0.012 ms/img	0.055 ms/img	4.7
F1-macro en prueba	0.864	0.789	-7.5 puntos

Hardware: Intel Core i7-1165G7 (2.8 GHz), 15.7 GB de RAM, sin GPU, Windows 11. Entorno: Python 3.12.13, TensorFlow 2.21.0 (CPU), Keras 3.15.1 y scikit-learn 1.9.1. La inferencia se midió sobre las 10,000 imágenes de prueba, en lotes de 256 y después de una pasada de calentamiento. Al repetir la medición en el notebook salieron 0.009 y 0.036 ms por imagen. En cuatro corridas del script, la CNN fue entre 8.4 y 11.5 veces más lenta al entrenar y entre 4 y 5 veces al inferir: los valores absolutos cambian, el orden no.

Decisión. En este benchmark, con este presupuesto, la CNN no justifica su costo. Tiene 2.6 veces menos parámetros y ocupa 262 KB frente a 619 KB, pero tarda unas 10 veces más en entrenar, es entre 4 y 5 veces más lenta en inferencia sobre CPU y obtiene 7.5 puntos menos de F1-macro. Con 30 épocas la brecha baja a 2.8 puntos, pero el entrenamiento de la CNN sube a 241 s, frente a 16 s del denso. Elijo el baseline denso. Menos parámetros no significa menor costo: las convoluciones hacen muchas más operaciones por imagen que una capa densa pequeña. La causa probable del bajo desempeño de la CNN es su cabeza, porque GlobalAveragePooling promedia la posición de los rasgos y la red tiene pocos filtros. Antes de descartar las convoluciones probaría una cabeza Flatten + Dense o más filtros y volvería a medir el costo. No evalué esa hipótesis en esta entrega.

6. Código, reproducibilidad y repositorio

Ruta	Contenido
src/inf8239_u02_cv/	data.py (clases, normalización, validación, partición, auditoría), models.py (build_dense, build_cnn) y evaluation.py (inferencia repetida, pares confundidos, tamaño de archivo)
scripts/train_cv.py	Entrena los dos modelos y genera las métricas, las figuras y los modelos
notebooks/01_fashion_mnist_cnn.ipynb	Notebook ejecutado: auditoría, carga de los modelos guardados, recálculo del F1 (coincide con el reporte), curvas, errores y costo
reports/, reports/epochs30/	cv_metrics.json, classification_reports.json, audit.json, history.json y las figuras de las secciones 4 y 5
models/	best_cnn.keras y dense_baseline.keras
README.md, MODEL_CARD.md	Instrucciones de ejecución y ficha del modelo
pyproject.toml, uv.lock, requirements.txt	Entorno fijado: uv (ruta principal), pip (requirements.txt, exportado del lock) y Colab (requirements-colab.txt)

Para reproducir todo desde la raíz del proyecto (detalle en el README):

```
uv python install 3.12
uv sync --extra cpu
uv run python scripts/check_runtime.py
uv run python scripts/train_cv.py --epochs 8
uv run python scripts/train_cv.py --epochs 30 --run-name epochs30
uv run pytest -q
uv run jupyter nbconvert --to notebook --execute --inplace notebooks/01_fashion_mnist_cnn.ipynb
```

El .gitignore excluye .venv y las cachés, pero no reports/ ni models/, para que la evidencia quede versionada (los dos modelos pesan menos de 1 MB). El historial de Git sigue los commits del laboratorio: *chore: initialize isolated computer vision project* y *feat: compare dense baseline and CNN*.

7. Pruebas automatizadas

Las pruebas usan pytest y se ejecutan con *uv run pytest -q* (pyproject.toml ya añade src al path). No necesitan descargar el dataset.

Archivo	Prueba	Qué comprueba
test_data.py	test_normalize_adds_channel_and_scales	Forma (n, 28, 28, 1) y rango [0, 1]
test_data.py	test_validate_rejects_wrong_shape	Rechaza imágenes sin canal
test_data.py	test_validate_rejects_values_outside_range	Rechaza píxeles sin normalizar
test_data.py	test_validate_rejects_unknown_label	Rechaza etiquetas fuera de 0 a 9
test_data.py	test_there_are_ten_classes	Hay 10 clases con nombres únicos
test_data.py	test_split_is_deterministic_and_disjoint	La partición es fija y sin solapamiento
test_data.py	test_audit_detects_overlap_between_splits	La auditoría detecta imágenes compartidas
test_model_contract.py	test_model_output_contract (CNN y denso)	Salida (n, 10), probabilidades finitas, no negativas y que suman 1
test_model_contract.py	test_cnn_output_contract	Contrato original del laboratorio
test_model_contract.py	test_cnn_has_fewer_parameters_than_dense	La CNN tiene menos parámetros que el denso

Resultado: las 11 pruebas pasan (11 passed en 5.6 s, pytest 8.4.2). El linter ruff no reporta errores en src, scripts ni tests.

8. Model Card

MODEL_CARD.md documenta los puntos que pide el laboratorio. Resumen:

Apartado	Contenido
Uso previsto	Ejercicio académico de clasificación de prendas en imágenes de 28×28 en escala de grises
Fuera de alcance	Fotos reales de ropa, imágenes médicas, OCR, vigilancia o cualquier decisión sobre personas
Datos	Fashion-MNIST: 54,000 / 6,000 / 10,000, auditado sin duplicados ni solapamiento; píxeles en [0, 1]
Métricas	F1-macro 0.789 (CNN) y 0.864 (denso), F1 por clase y pares más confundidos
Costo	Hardware, versiones, parámetros, tamaño, tiempo de entrenamiento e inferencia
Riesgos	Shirt con recall de 0.36, errores con confianza alta, posibles etiquetas erróneas, una sola semilla
Supervisión	Revisión humana obligatoria; antes de cambiar de dominio, reentrenar con datos propios y revisar métricas por clase

9. Conclusión

Pregunta del cierre	Respuesta
Resultado principal	Con la misma partición, el baseline denso (F1-macro 0.864) superó a la CNN pequeña (0.789).
Evidencia predictiva	El denso tiene mayor F1 en las 10 clases y con 30 épocas (0.875 frente a 0.848). Las curvas muestran que la CNN subajusta.
Clase más difícil	Shirt: F1 de 0.42 y recall de 0.36 en la CNN. Se confunde con T-shirt/top, Coat y Pullover.
Costo comparado	La CNN tiene 2.6 veces menos parámetros y un archivo de 262 KB, pero tarda unas 10 veces más en entrenar y es entre 4 y 5 veces más lenta en inferencia sobre CPU.
¿La mejora justifica el costo?	No hay mejora que justificar: la CNN cuesta más y rinde menos. Elijo el denso para este benchmark.
Limitación del benchmark	Imágenes centradas, de 28×28, en grises y con fondo limpio. Hay una sola semilla y algunas etiquetas dudosas. El resultado no se transfiere a otros dominios.
Decisión antes de usar otro dominio	Recolectar y etiquetar datos propios, repetir la auditoría y la partición, probar una CNN con más capacidad y evaluar por clase con revisión humana.

Lo que más aprendí no fue el número final. Una CNN no gana automáticamente: con poca capacidad y pocas épocas, una red densa sencilla es mejor y más barata. Sin las curvas habría confundido subajuste con un límite del modelo, y sin las métricas por clase la accuracy de 0.79 a 0.87 habría ocultado que la CNN solo reconoce una de

cada tres camisas.

10. Uso de inteligencia artificial

Herramienta	Claude Code (Claude Opus 5.5, Anthropic) en Visual Studio Code, octubre de 2026.
Mi participación	Preparé el entorno con uv, ejecuté el entrenamiento inicial y las pruebas del proyecto base, e hice el commit inicial. Verifiqué el informe y pedí correcciones que se aplicaron, como guardarlo en reports/.
Prompts (resumen)	Revisar el proyecto, luego de haber corrido yo los scripts del laboratorio. Pedí compararlo con el mandato y la rúbrica, apoyarme en lo que faltara y explicarme cada punto; completar la evidencia faltante con resultados reales.
Tareas asistidas	Ampliar train_cv.py, crear evaluation.py, el notebook y 8 pruebas, y proponer el README, el Model Card y el borrador del informe.
Verificación	Las cifras salen de reports/*.json. Mi corrida inicial y las cuatro posteriores dieron el mismo F1; el notebook lo recalcula con los modelos guardados. Las 11 pruebas pasan.
Correcciones	Se corrigió el dato de una prueba que fallaba, se dejó de excluir reports/ y models/ en .gitignore, se añadió el experimento de 30 épocas en lugar de ajustar el resultado, se agrandaron los rótulos de la figura de errores y se movió el informe a reports/.
Responsabilidad	Asumo la responsabilidad de lo entregado.

11. Fuentes

Abadi, M., et al. (2016). *TensorFlow: Large-scale machine learning on heterogeneous distributed systems*. arXiv. https://doi.org/10.48550/arXiv.1603.04467

Mitchell, M., et al. (2019). Model cards for model reporting. *Proceedings of FAT* 2019*, 220-229. https://doi.org/10.1145/3287560.3287596

Pedregosa, F., et al. (2012). *Scikit-learn: Machine learning in Python*. arXiv. https://doi.org/10.48550/arXiv.1201.0490

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms*. arXiv. https://doi.org/10.48550/arXiv.1708.07747